Advanced Programming Techniques
CA1 Report

1. Introduction

This assignment focuses on applying practical programming skills using both C# and Python. The tasks are designed to cover different areas of programming such as object-oriented programming, data structures, network communication, and web scraping. Each task builds on fundamental concepts and applies them in a simple and structured way.

The goal of this assignment is not only to produce working programs but also to understand how these concepts work together in real-world scenarios. The solutions were developed with a focus on clarity, simplicity, and correctness so that they are easy to understand and maintain.

2. Task 1: Churros Shop Management System

Task 1 required the development of a simple console-based application to simulate a churros shop ordering system. The program was developed using C# and follows object-oriented programming principles.

The system uses classes such as Order and Churros to represent real-world entities. Each order contains information such as flavor, quantity, and total price. A queue data structure is used to manage orders. This ensures that orders are processed in a First In First Out (FIFO) manner, which reflects how real shops operate.

Encapsulation is applied through properties. For example, the OrderNo property is controlled using a private setter, which ensures that it cannot be modified from outside the class. Constructors are used to initialize objects when they are created.

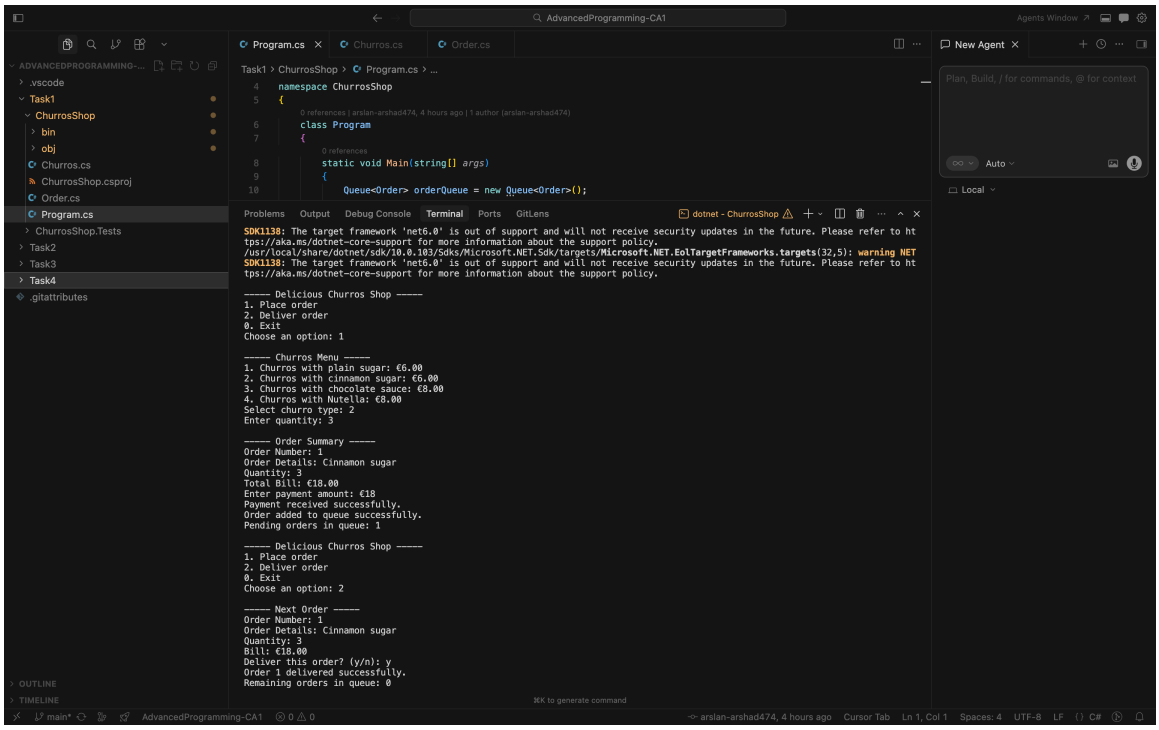


Figure 1: Task 1-Program output showing menu, order placement, and delivery

A unit test was implemented to test the PayBill() method. This ensures that the calculation of the bill is correct. Testing improves reliability and helps detect errors early.

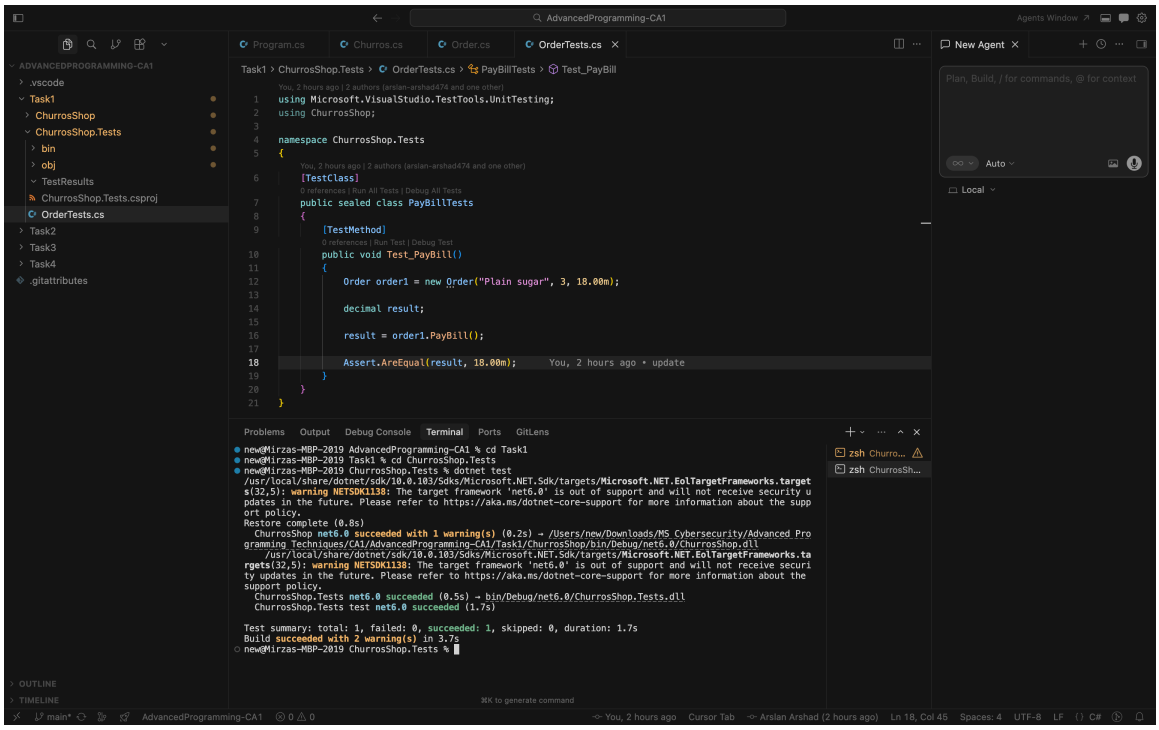


Figure 2: Task 1-Unit test result for PayBill()

Overall, the program is structured, uses proper OOP concepts, and performs all required operations successfully.

3. Task 2: Periodic Table Information System

Task 2 involved creating a C# program to store and retrieve information about the first 30 elements of the periodic table. A list data structure was used to store element objects. Each element contains attributes such as atomic number, name, classification, and description. The user inputs an atomic number, and the program retrieves the corresponding element from the list.

The program includes input validation to ensure that only numbers between 1 and 30 are accepted. If the user enters an invalid value, the program displays an error message and asks again.

A loop is used to allow the user to continue searching for elements until they choose to exit. This improves usability and matches the example execution provided in the assignment.

This task demonstrates how data can be stored in structured objects and accessed efficiently using indexing. The program is simple, clean, and easy to follow.

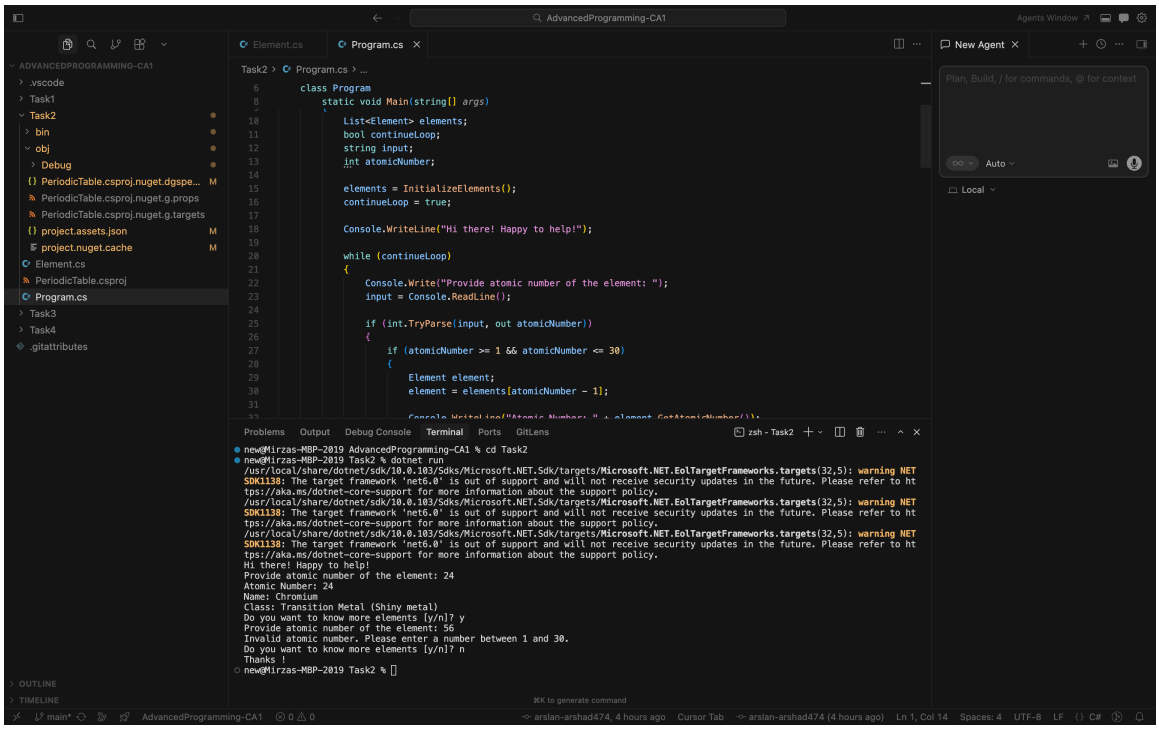


Figure 3: Task 2-Output showing element details for a valid atomic number

4. Task 3: TCP Client-Server Communication

Task 3 was completed in Python and focuses on socket programming using TCP. Two separate files were created: one for the server and one for the client. The server waits for a connection on localhost and port 9999. Once the client connects, it sends data to the server, and the server responds, using a TCP_server() function, a TCP_client() function, a main() function, and try-except blocks for error handling. The task also includes storing customer details in an SQLite database and returning a unique registration number from the server back to the client. This task was useful for understanding how two systems communicate over a network and how Python can combine socket programming with simple database storage.

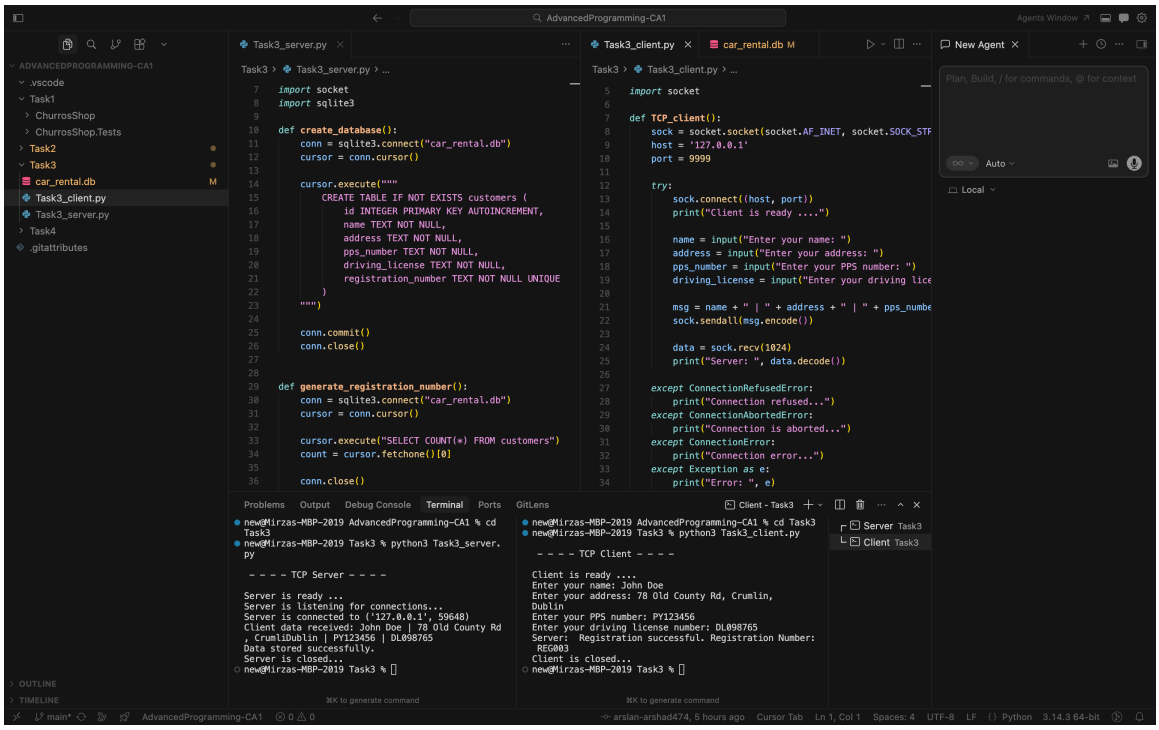


Figure 4: Task 3-Server terminal showing connection status and client terminal showing data sent and registration number returned

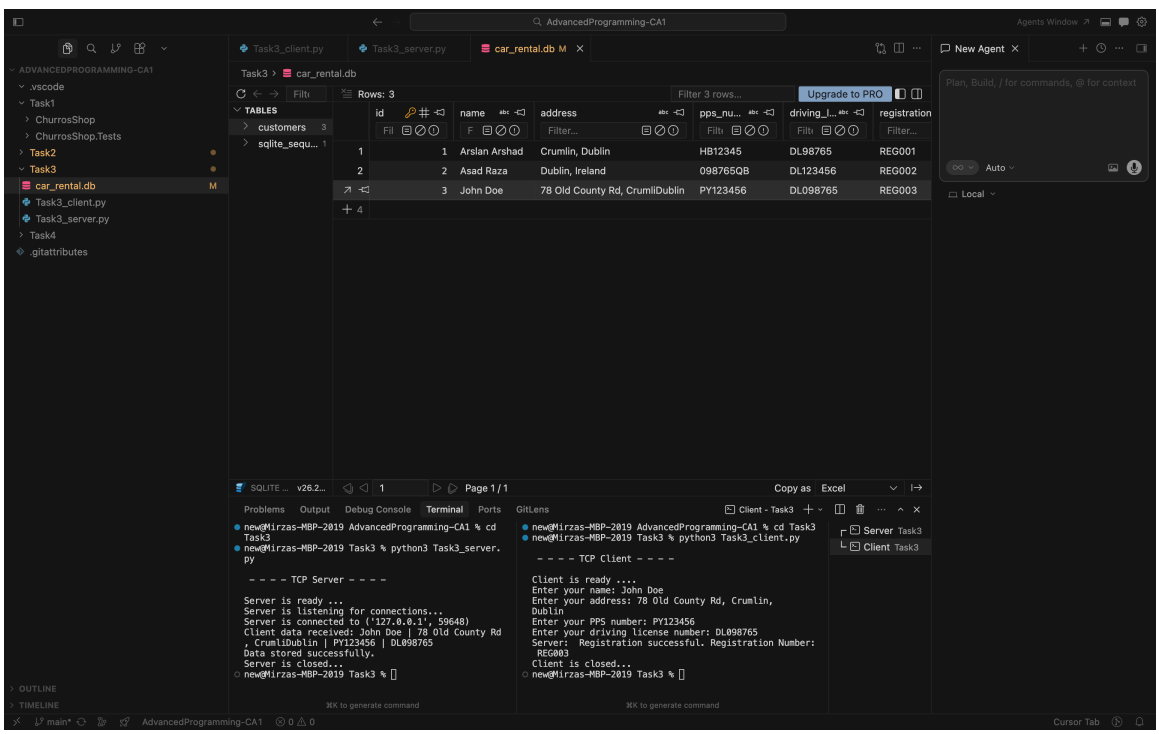


Figure 5: Task 3-Showing data in database

5. Task 4: Web Scraping and CSV Handling

Task 4 required scraping data from a website and storing it in a CSV file using Python. The requests library was used to fetch the webpage, while BeautifulSoup was used to parse the HTML content. The program targets specific HTML elements such as article with class “product_pod” to extract book data.

For each book, the following information was extracted:

1. Title
2. Price
3. Rating

The extracted data was then stored in a CSV file using Python’s csv module. This allows the data to be saved in a structured format.

At the end of the program, the CSV file is read again and displayed in the terminal. This confirms that the data was stored correctly.

One issue encountered was the presence of the “Â” character before the price. This occurs due to encoding issues when reading special characters like the pound symbol. This was resolved by cleaning the text output.

This task demonstrates how data can be collected from websites and stored for further use.

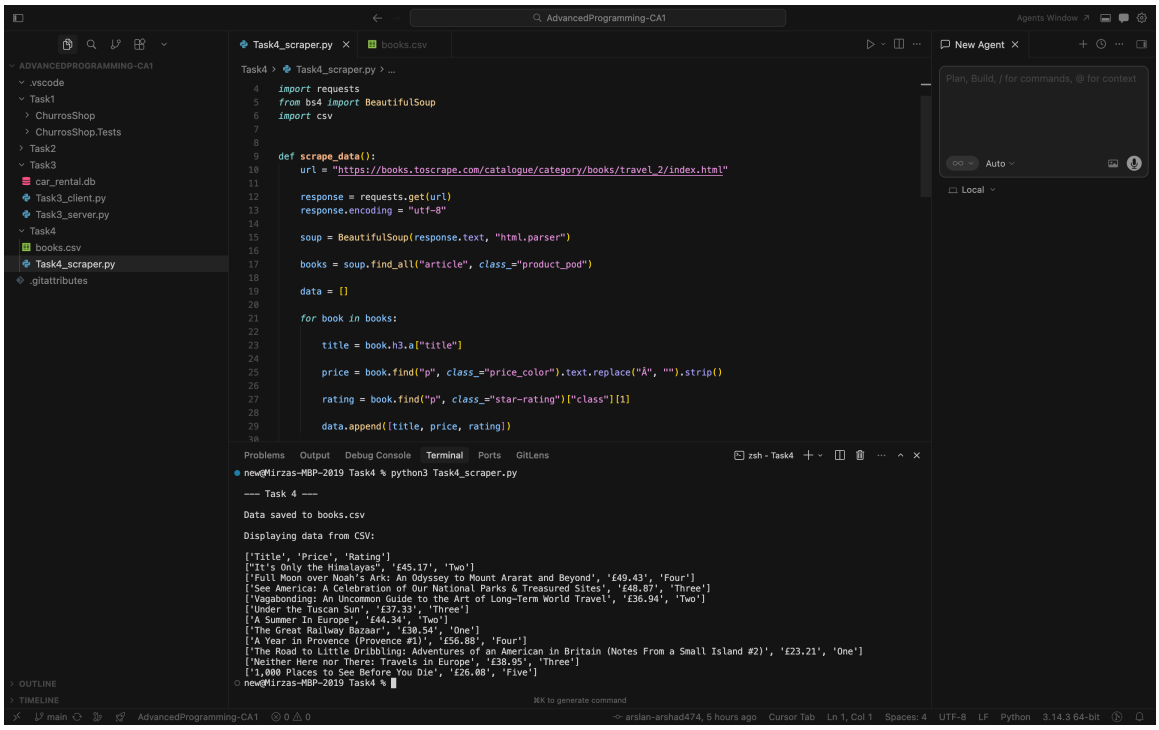
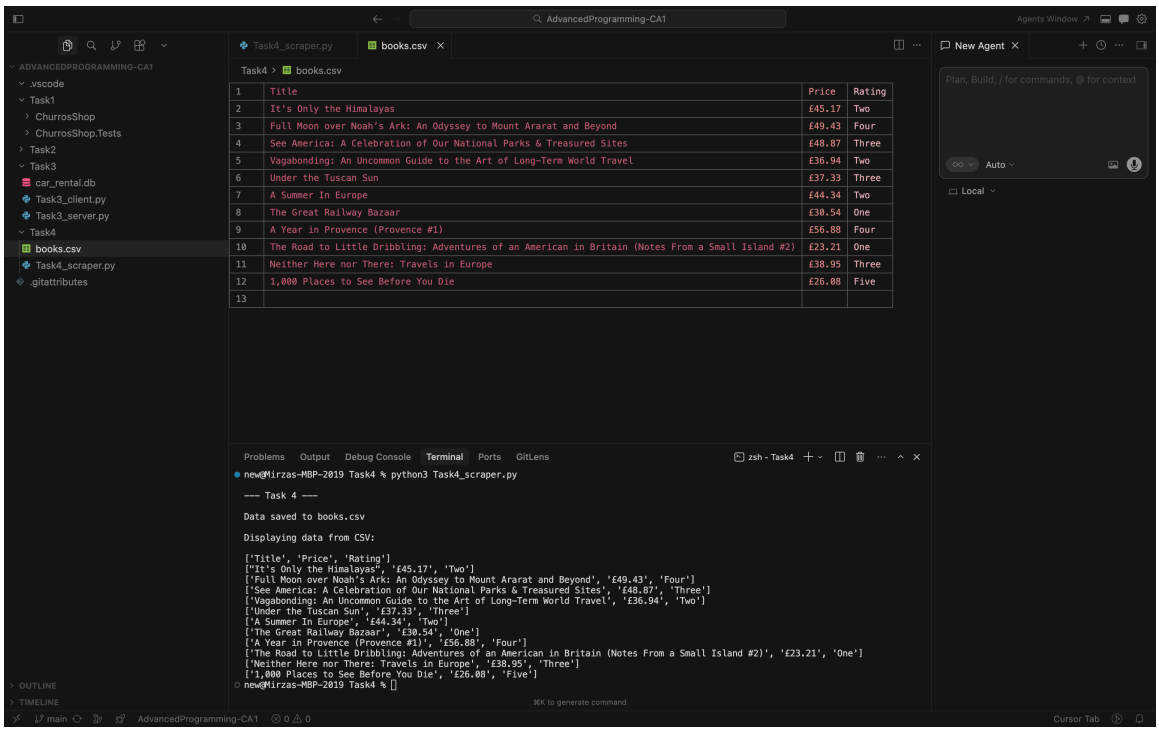


Figure 6: Task 4-Terminal output showing scraped book data



	Title	Price	Rating
1	Title	Price	Rating
2	It's Only the Himalayas	£45.17	Two
3	Full Moon over Noah's Ark: An Odyssey to Mount Ararat and Beyond	£49.43	Four
4	See America: A Celebration of Our National Parks & Treasured Sites	£48.87	Three
5	Vagabonding: An Uncommon Guide to the Art of Long-Term World Travel	£36.94	Two
6	Under the Tuscan Sun	£37.33	Three
7	A Summer In Europe	£44.34	Two
8	The Great Railway Bazaar	£30.54	One
9	A Year in Provence (Provence #1)	£56.88	Four
10	The Road to Little Dribbling: Adventures of an American in Britain (Notes From a Small Island #2)	£23.21	One
11	Neither Here nor There: Travels in Europe	£38.95	Three
12	1,000 Places to See Before You Die	£26.08	Five
13			

Figure 7: Task 4-Books.csv file showing stored scraped data

6. Challenges Faced

Several challenges were faced during the assignment. In Task 1, setting up unit tests caused errors due to missing dependencies. This was resolved by properly configuring the test project. In Task 2, ensuring correct property names and structure was important to avoid runtime errors. In Task 3, connection issues occurred when the server was not running before the client. In Task 4, understanding the HTML structure of the webpage was initially confusing. However, inspecting the page using browser developer tools helped identify the correct tags and classes.

These challenges helped improve debugging skills and understanding of the concepts.

7. GitHub Repository and Conclusion

The coursework files were uploaded to GitHub so that progress and final code could be stored in one place. The repository link for this assignment is: https://github.com/arslan-arshad474/AdvancedProgramming-CA1. Including the repository supports version control and shows the development record of the work. Overall, this assignment was useful because it combined several important programming areas in one submission. Task 1 strengthened understanding of OOP and queues, Task 2 improved data handling and structured access to information, Task 3 introduced networking and database integration, and Task 4 demonstrated scraping and file output.

References

Beautiful Soup Documentation (2024). Available at: https://www.crummy.com/software/BeautifulSoup/bs4/doc/

Requests Library (2024). Available at: https://docs.python-requests.org/

W3Schools Python CSV (2024). Available at: https://www.w3schools.com/python/python_csv.asp

Books to Scrape (2024). Available at: https://books.toscrape.com/

Microsoft .NET Documentation (2024). Available at: https://learn.microsoft.com/

PubChem (2024). Available at: https://pubchem.ncbi.nlm.nih.gov/element/3